

Index		
Matplotlib	- 1	Pandas -7
M.1 Line Plot	- 1	P.1 Read CSV -7
M.2 Scatter Plot	- 2	P.2 Get Dummies -7
M.3 Histogram	- 2	P.3 Select data types -8
M.4 Bar Graph	- 3	P.4 Drop Column -8
M.5 Labels & Title	- 3	P.5 Fill null -9
M.6 Subplots	- 4	P.6 Group by/ Median -9
M.7 Figure Size	- 4	P.7 Mode -10
M.8 Tight Layout	- 5	P.8 Is Null -10
M.9 Add Text	- 5	P.9 Sum -10
M.10 Legend	- 6	P.10 Duplicate -11
Seaborn	- 6	P.11 Drop Duplicates -11
SB.1 Boxplot	- 6	P.12 Unique -11
Numpy	- 16	P.13 Sort Values -12
N.1 Log	- 16	P.14 Copy -12
N.2 Log1p	- 16	P.18 Map/Replace -14
N.3 Exp	- 17	P.21 Describe -15
N.4 Expm1	- 17	P.22 Value Counts -15
N.5 Sqrt	- 17	P.23 Select Training -15
		Scikit - 19
		S.1 Linear Regression - 19
		S.2 Ridge Regression - 19
		S.3 Logistic Regression - 20
		S.4 Random Forest - 21
		S.5 Gradient Boosting - 21
		S.6 R2 Score - 22
		S.7 Mean Squared Error - 22

ML Reference Sheet

Matplotlib

Import Syntax:

```
Python  
import matplotlib.pyplot as plt
```

M.1 plt.plot()

Syntax:

```
Python  
plt.plot(x, y)
```

Parameters:

- x: Data for x-axis.
- y: Data for y-axis.

Description: Creates a basic line plot to show trends over time or sequences.

Example:

```
Python  
plt.plot([1, 2, 3], [4, 5, 6])
```

M.2 plt.scatter()

Syntax:

```
Python  
plt.scatter(x, y)
```

Parameters:

- x: Data for x-axis.
- y: Data for y-axis.

Description: Creates a scatter plot to visualize relationships or distributions between two variables.

Example:

```
Python  
plt.scatter([1, 2, 3], [4, 5, 6])
```

M.3 plt.hist()

Syntax:

```
Python  
plt.hist(x, bins=10)
```

Parameters:

- x: Numeric array to plot.
- bins: The number of intervals or bins to divide the data.

Description: Generates a histogram to visualize the distribution of numerical data.

Example:

```
Python  
plt.hist([1, 2, 2, 3], bins=3)
```

M.4 plt.bar()

Syntax:

```
Python  
plt.bar(x, height)
```

Parameters:

- x: Categories for the x-axis.
- height: The height of the bars representing the value.

Description: Creates a bar graph to compare different categories.

Example:

```
Python  
plt.bar(['A', 'B'], [10, 20])
```

M.5 plt.xlabel() / plt.ylabel() / plt.title()

Syntax:

```
Python  
plt.xlabel(label), plt.ylabel(label), plt.title(title)
```

Description: Sets the label for the x-axis, y-axis, and the main title of the plot.

Example:

```
Python  
plt.xlabel("X-Axis")
```

M.6 plt.subplots()

Syntax:

Python

```
fig, ax = plt.subplots(nrows=1, ncols=1)
```

Parameters:

- nrows: Number of rows of subplots.
- ncols: Number of columns of subplots.

Description: Creates a figure and a set of subplots to display multiple graphs in one canvas.

Example:

Python

```
fig, ax = plt.subplots()
```

M.7 plt.figure()

Syntax:

Python

```
plt.figure(figsize=(width, height))
```

Description: Sets the figure size before the plot command to ensure graphs are readable and not squeezed.

Example:

Python

```
plt.figure(figsize=(10, 6))
```

M.8 plt.tight_layout()

Syntax:

Python

```
plt.tight_layout()
```

Description: Automatically adjusts subplots so titles, axis labels, and numbers don't overlap.

Example:

Python

```
plt.tight_layout()
```

M.9 plt.text()

Syntax:

Python

```
plt.text(x_coord, y_coord, "Text String")
```

Description: Places labels inside the plot space, frequently used for overlaying metrics like R^2 or RMSE.

Example:

Python

```
plt.text(2, 50, "R2 = 0.95")
```

M.10 plt.legend()

Syntax:

```
Python  
plt.legend()
```

Description: Renders a key on the chart to identify variables.

Example:

```
Python  
plt.legend()
```

Seaborn

Import Syntax:

```
Python  
import seaborn as sns
```

SB.1 sns.boxplot()

Syntax:

```
Python  
sns.boxplot(x=None, y=None, data=None)
```

Parameters:

- x: The categorical column string representing the categories along the bottom axis.
- y: The continuous numeric column string plotted vertically.
- data: The DataFrame containing the specified column names.

Description: Generates a box-and-whisker plot to visually check median steps and find outlier spikes.

Example:

```
Python  
sns.boxplot(x='category', y='value', data=df)
```

Pandas

Import Syntax:

```
Python
import pandas as pd
```

P.1 pd.read_csv()

Syntax:

```
Python
pd.read_csv(filepath_or_buffer, ...)
```

Parameters:

filepath_or_buffer: The path or URL of the file to be read.

Description: Loads a comma-separated values (CSV) file into a Pandas DataFrame.

P.2 pd.get_dummies()

Syntax:

```
Python
pd.get_dummies(data, columns=None, drop_first=False)
```

Parameters:

- data: The DataFrame you want to process.
- columns: A list of specific column names to encode. If None, encodes all string columns.
- drop_first: Boolean. If True, drops the first category dummy column to avoid multicollinearity.

Description: Performs One-Hot Encoding. Converts text categories into separate columns of 0s and 1s.

Example:

```
Python
pd.get_dummies(df['column_name'])
```

P.3 df.select_dtypes()

Syntax:

Python

```
df.select_dtypes(include=None, exclude=None)
```

Parameters:

- include: A list of data types or strings (e.g., ['int64', 'float64']) to keep.
- exclude: A list of data types or strings (e.g., ['object']) to completely drop.

Description: Filters the DataFrame columns to return a subset based on their pixel/data types.

Example:

Python

```
df.select_dtypes(include=[ 'float64' ])
```

P.4 df.drop()

Syntax:

Python

```
df.drop(columns=None, errors='ignore')
```

Parameters:

- columns: A single column string or a list of column names you want to delete.
- errors: If set to 'ignore', it prevents the code from crashing if a column doesn't exist.

Description: Removes specified rows or columns completely from a DataFrame.

Example:

Python

```
df.drop(columns=[ 'col_1', 'col_2' ])
```

P.5 df.fillna()

Syntax:

```
Python  
df.fillna(value=None)
```

Parameters:

- value: The scalar value (like 0 or 'None') used to replace all missing (NaN) cells.

Description: Replaces missing (NaN) values with a specified value (the mode in this case).

Example:

```
Python  
df.fillna(0)
```

P.6 df.groupby() & .median()

Syntax:

```
Python  
df.groupby(by) ['Target_Column'].median().sort_values()
```

Parameters:

- by: The categorical column name you want to group your data rows by (e.g., 'Neighborhood').

Description: Splits data into groups, computes the middle value (median) for each group, and sorts them.

Example:

```
Python  
df.groupby('category')['value'].median()
```

P.7 df.mode()

Syntax:

```
Python  
df['column'].mode()
```

Description: Computes the most frequent value(s) in a column to fill missing data.

Example:

```
Python  
df['col'].mode()
```

P.8 df.isnull()

Syntax:

```
Python  
df.isnull()
```

Description: Checks for missing values across the DataFrame.

Example:

```
Python  
df.isnull()
```

P.9 df.sum()

Syntax:

```
Python  
df.sum()
```

Description: Sums up boolean indicators (used after .isnull() and .duplicated()).

Example:

```
Python  
df.isnull().sum()
```

P.10 df.duplicated()

Syntax:

```
Python  
df.duplicated()
```

Description: Identifies duplicate rows in the dataset.

Example:

```
Python  
df.duplicated()
```

P.11 df.drop_duplicates()

Syntax:

```
Python  
df.drop_duplicates()
```

Description: Removes duplicate rows from the DataFrame.

Example:

```
Python  
df.drop_duplicates()
```

P.12 df.unique()

Syntax:

```
Python  
df['column'].unique()
```

Description: Returns the unique values present in a specific column.

Example:

```
Python  
df['column'].unique()
```

P.13 df.sort_values()

Syntax:

Python

```
df.sort_values(by='column', ascending=False)
```

Description: Sorts the DataFrame (used to sort the coefficient DataFrame in descending order).

Example:

Python

```
df.sort_values(by='col', ascending=False)
```

P.14 df.copy()

Syntax:

Python

```
df.copy()
```

Description: Creates a brand new, isolated physical copy of the DataFrame to avoid SettingWithCopy Warnings.

Example:

Python

```
df_copy = df.copy()
```

P.15 df.head()

Syntax:

Python

```
df.head(n=5)
```

Parameters:

- n: Number of rows to select from the top.

Description: Returns the first n rows of the DataFrame (great for a quick visual check).

P.16 df.info()

Syntax:

```
Python  
df.info()
```

Description: Prints a concise summary of a DataFrame, including index data type, column data types, non-null values, and memory usage.

Example:

```
Python  
df.info()
```

P.17 df.shape

Syntax:

```
Python  
df.shape
```

Description: A property that returns a tuple representing the dimensionality of the DataFrame: (rows, columns).

Example:

```
Python  
rows, cols = df.shape
```

Attributes:

- df.columns – Returns the column labels of the DataFrame (used to align features with their coefficients).
- df.shape – Returns a tuple representing the dimensionality (rows, columns) of the DataFrame.

Description: DataFrame attributes for accessing metadata like columns and shape.

P.18 .map() or .replace()

Syntax:

Python

```
df['column'].map({'Low': 0, 'Medium': 1, 'High': 2})
```

Description: Essential for Ordinal Encoding; maps categories to meaningful numeric hierarchies.

Example:

Python

```
df['Quality'].map({'Poor': 0, 'Good': 1})
```

P.19 .str Accessors

Syntax:

Python

```
df['column'].str.lower()
```

Description: Essential for cleaning messy text strings (fixing inconsistent capitalization or whitespace).

Example:

Python

```
df['Name'].str.strip()
```

P.20 Boolean Indexing / Filtering

Syntax:

Python

```
df[df['column'] > value]
```

Description: Used for filtering outliers or creating targeted subsets.

P.21 .describe()

Syntax:

```
Python  
df.describe()
```

Description: Generates quick summary statistics (mean, min, max, quartiles) for numeric attributes.

Example:

```
Python  
df.describe()
```

P.22 .value_counts()

Syntax:

```
Python  
df['column'].value_counts(dropna=False)
```

Description: Crucial for checking class balances in classification targets or distributing categorical values.

Example:

```
Python  
df['Neighborhood'].value_counts()
```

P.23. Selecting Columns for Training

Syntax:

```
Python  
X = df[selected_features].values  
y = df['SalePrice'].values
```

Description: Extracts specific columns from a DataFrame to create feature (X) and target (y) matrices for model training, converting them into NumPy arrays.

NumPy

Import Syntax:

Python

```
import numpy as np
```

N.1 np.log()

Syntax:

Python

```
np.log(x)
```

Description: Calculates the natural logarithm. Used to compress heavily skewed target price variables.

Example:

Python

```
np.log(df['price'])
```

N.2 np.log1p()

Syntax:

Python

```
np.log1p(x)
```

Description: Calculates $\log(x + 1)$. Crucial for feature columns because it safely avoids crashing on 0 values.

Example:

Python

```
np.log1p(df['price'])
```

N.3 np.exp()

Syntax:

```
Python  
np.exp(x)
```

Description: The inverse of log(). Converts compressed log values back into real-world dollar amounts.

Example:

```
Python  
np.exp(df['log_price'])
```

N.4 np.expm1()

Syntax:

```
Python  
np.expm1(x)
```

Description: Calculates the exponential of all elements in the input array minus 1. This completely reverses a np.log1p transformation.

Example:

```
Python  
np.expm1(df['log_price'])
```

N.5 np.sqrt()

Syntax:

```
Python  
np.sqrt(x)
```

Description: Computes the non-negative square root of an array or single value element-wise.

Example:

```
Python  
np.sqrt(df['data'])
```

Scikit-Learn

Module: sklearn.model_selection

Import Syntax:

Python

```
from sklearn.model_selection import train_test_split
```

Parameters:

- `*arrays`: The X (features) and y (target) arrays you want to slice up.
- `test_size`: The float ratio representing the proportion of the dataset to hold out (e.g., 0.2 = 20%).
- `random_state`: Sets a static seed for the shuffle so your train/test split is perfectly reproducible.

Description: Splits your data matrices into random train and test subsets to evaluate generalization.

Example:

Python

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2)
```

Module: sklearn.linear_model

Import Syntax:

```
Python
from sklearn.linear_model import LinearRegression, Ridge,
LogisticRegression
```

S.1 LinearRegression()

Syntax:

```
Python
LinearRegression()
```

Description: Fits a standard, unpenalized ordinary least squares line. Draws a straight trend line.

Example:

```
Python
model = LinearRegression().fit(X, y)
```

Description: Model for linear regression tasks.

S.2 Ridge()

Syntax:

```
Python
Ridge(alpha=1.0)
```

Parameters:

- alpha: Regularization strength. Larger values shrink coefficients closer to zero to control overfitting.

Description: Linear regression with an L2 penalty. Prevents overfitting when handling hundreds of columns.

Example:

```
Python
model = Ridge(alpha=1.0).fit(X, y)
```

S.3 LogisticRegression()

Syntax:

Python

```
LogisticRegression(class_weight=None, random_state=None)
```

Parameters:

- `class_weight`: Weights associated with classes. 'balanced' automatically adjusts weights to handle imbalance.

Description: Logistic Regression classifier estimator object used to predict categorical outcomes.

Example:

Python

```
model = LogisticRegression().fit(X, y)
```

Module: sklearn.ensemble

Import Syntax:

Python

```
from sklearn.ensemble import RandomForestRegressor,  
HistGradientBoostingRegressor
```

S.4 RandomForestRegressor()

Syntax:

Python

```
RandomForestRegressor(n_estimators=100, random_state=None)
```

Parameters:

- `n_estimators`: The total number of independent decision trees to build inside the forest.
- `random_state`: Controls randomness to ensure you get the exact same score every time you hit run.

Description: An ensemble model that constructs hundreds of independent decision trees and averages them.

Example:

Python

```
model = RandomForestRegressor().fit(X, y)
```

S.5 HistGradientBoostingRegressor()

Syntax:

Python

```
HistGradientBoostingRegressor(random_state=42)
```

Description: An ensemble model that builds trees sequentially. Each tree explicitly corrects the errors of the last.

Example:

Python

```
model = HistGradientBoostingRegressor().fit(X, y)
```

Module: sklearn.metrics

Import Syntax:

Python

```
from sklearn.metrics import r2_score, mean_squared_error
```

S.6 r2_score()

Syntax:

Python

```
r2_score(y_true, y_pred)
```

Parameters:

- `y_true`: The actual, ground-truth real estate values from your test split.
- `y_pred`: The mathematical guesses generated by your trained model.

Description: Calculates the R-squared metric (0% to 100%), measuring the proportion of variance predicted.

Example:

Python

```
score = r2_score(y_true, y_pred)
```

S.7 mean_squared_error()

Syntax:

Python

```
mean_squared_error(y_true, y_pred)
```

Description: Measures the average squared difference between estimated values and the actual target value.

Example:

Python

```
mse = mean_squared_error(y_true, y_pred)
```